

Calorie Tracker Website

Antigravity / AI Coding Agent - Master Phased Build Prompt

A mobile-first nutrition tracking web application where users calculate maintenance calories, search foods by name, choose a practical serving quantity, view calories/macros/micros, log meals, see daily calorie progress, and return to previous days through a calendar.

IMPORTANT EXECUTION RULE

Run only the phase explicitly requested by the user. After that phase is complete, update README.md and EXPLANATION.md, verify the phase, summarize changes, identify any external account/action needed from the user, and STOP. Never silently start the next phase.

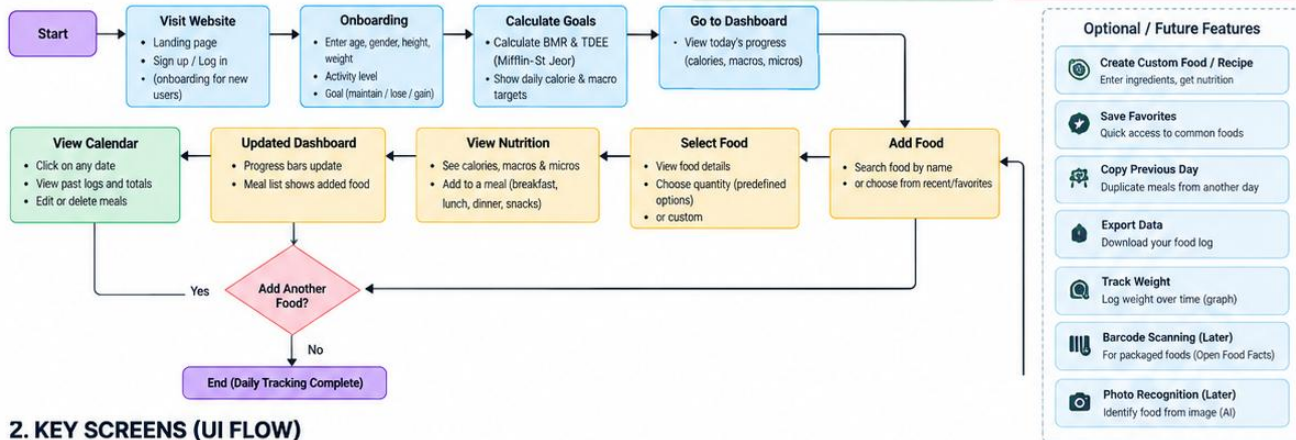
Calorie Tracker Website

Track your nutrition. Build healthier habits.

Search food, select quantity, get calories, macros & micros, track daily progress, and view past days on a calendar.

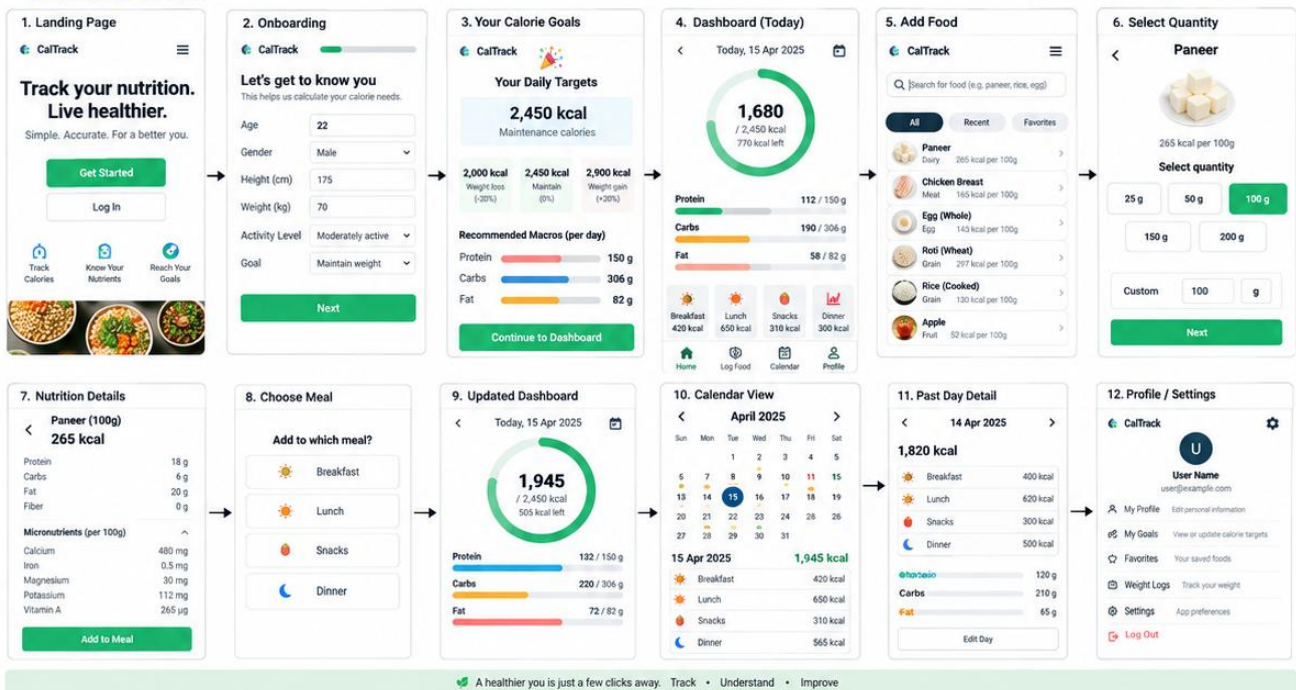
1. USER FLOW DIAGRAM

Complete flow of how the website works



2. KEY SCREENS (UI FLOW)

Main screens the user will see



Reference UI / flow concept. The implementation should follow the information architecture and visual direction, not copy any particular text, colors, or artwork literally.

MASTER PROMPT TO GIVE ANTIGRAVITY

HOW TO USE THIS DOCUMENT

Copy the Master Prompt section into Antigravity. Then start by saying: "Execute Phase 0 only." For later phases, say "Execute Phase 1 only", etc. Keep the numbered phase structure unchanged so progress stays traceable.

You are the lead software engineer, product designer, QA engineer, and technical writer for a mobile-first web application called Calorie Tracker. Build the application incrementally in strict phases. Do not skip phases, do not

implement future features early, and do not make architectural changes without documenting them. The user will explicitly tell you which phase to execute.

NON-NEGOTIABLE RULES FOR EVERY PHASE

1. Before changing code, inspect the existing repository and current implementation. Reuse what already works. Do not delete working code merely to make your implementation easier.
2. Execute ONLY the phase requested. Do not begin the next phase automatically.
3. At the start of each phase, state the goal, files/components likely to change, and any external dependency you may need.
4. If an external service, account, secret, API key, database project, deployment project, OAuth provider, domain, or paid service is required, STOP before using it. Clearly explain what is needed, why it is needed, exactly where the user must create it, and what non-secret configuration value can be shared. Never ask the user to paste a private secret into chat if it can be added locally through an .env file or dashboard.
5. Never invent API keys, database credentials, URLs, IDs, or external-service responses.
6. Keep secrets in environment variables. Provide or update .env.example with variable names and safe descriptions, never real secrets.
7. After each phase, run the relevant linting, type checking, unit tests, integration tests, and/or build checks that the repository supports. Fix errors before declaring the phase complete.
8. After each phase, update README.md and EXPLANATION.md. README.md must explain setup, commands, current features, environment variables, and phase status. EXPLANATION.md must explain how the website works in simple language, what each major component does, how data moves through the system, why architectural decisions were made, and how the AI coding agent implemented the phase.
9. Keep documentation easy to understand for a developer who did not build the project. Explain technical words the first time they appear.
10. Maintain the same visual language throughout the project: clean mobile-first cards, generous spacing, rounded corners, white/off-white surfaces, a green health-focused primary accent, restrained blue/teal supporting accents, readable typography, clear hierarchy, subtle borders/shadows, and accessible contrast.
11. Use responsive layouts first for mobile screens, then scale up gracefully for tablets and desktop. The phone experience is the primary design target.
12. Use accessible controls, semantic HTML, keyboard navigation where applicable, visible focus states, meaningful labels, and sensible loading/empty/error states.
13. Prefer simple maintainable code over unnecessary abstraction. Do not introduce libraries unless they solve a real need. When a new library is added, update documentation and explain why.
14. For nutrition calculations, treat database values as the source of truth. Do not use an LLM to invent nutrition numbers. Store nutrition values with source/attribution metadata where appropriate.
15. All nutrition values shown to users should be presented as estimates/standardized database values where recipe or brand variation can materially affect results. Do not market the application as providing medical advice.
16. When a task is ambiguous, choose the safest and simplest implementation consistent with this specification and document the assumption instead of blocking progress with unnecessary questions.
17. At the end of each phase, provide a concise completion report with: what was built, files changed, verification performed, screenshots or visual checks if available, known limitations, external actions required from the user, and the exact next phase that can be requested. Then STOP.

PRODUCT REQUIREMENTS

Core user journey

- New user opens the website and creates an account or starts onboarding.
- User enters age, gender, height, weight, activity level, and goal.

- The app calculates BMR and estimated maintenance calories (TDEE), then creates daily calorie and macro targets.
- User reaches the dashboard and sees calories consumed, calories remaining, calorie progress, macro progress, and today's logged meals.
- User taps Add Food, searches by food name, chooses a food, chooses from predefined quantities or a custom quantity, reviews nutrition, chooses a meal, and saves it.
- The dashboard immediately updates the consumed calories, remaining calories, macro progress, and meal totals.
- User opens Calendar to go backward to any logged day and view or edit the meals and nutrition for that date.
- User can update profile and goals later. Historical entries must remain tied to their original dates.

Primary navigation

- Home: today's nutrition summary and progress.
- Log / Add Food: search and log food.
- Calendar: browse daily history and monthly summaries.
- Profile: personal details, goals, favorites, custom foods, settings.

Dashboard requirements

- Large daily calorie progress visualization showing consumed versus target.
- Numeric calories consumed, target, and remaining. When over target, show the overage clearly rather than a broken/overflowing component.
- Progress bars for protein, carbohydrates, and fat against daily targets.
- Meal cards for breakfast, lunch, snacks, and dinner, each showing calories and allowing drill-down.
- Primary Add Food action that is always easy to reach on mobile.
- Loading, empty, error, and first-day states.

Calendar requirements

- Month navigation with previous/next controls and a Today shortcut.
- Tap any date to open a read-only or editable daily detail view depending on implementation stage.
- Visual indication of days with logged data and optional target-adherence status.
- Daily detail must show total calories, macro totals, meals, and the same calorie progress visualization used on the dashboard.
- Calendar data must be fetched by user and date range efficiently; avoid loading the entire lifetime history when one month is needed.

TECH STACK AND ARCHITECTURE

Layer	Preferred technology	Purpose
Frontend	Next.js + TypeScript	Mobile-first application UI, routing, server/client components where appropriate.
Styling	Tailwind CSS	Responsive layout and consistent design tokens.
UI	shadcn/ui or similarly accessible component primitives	Buttons, dialogs, inputs, cards, tabs, menus, etc.
Charts / visualizations	Recharts or lightweight CSS/SVG components	Nutrition trends where needed; prefer CSS/SVG for simple progress bars and

		rings.
Calendar	react-day-picker or equivalent accessible calendar	Monthly navigation and date selection.
Backend	FastAPI + Python	API layer, nutrition calculations, data validation, future integration points.
Database	PostgreSQL	Users, food catalog, nutrition facts, servings, meals, meal items, favorites, daily targets, weight logs.
Backend platform	Supabase	Managed PostgreSQL, Auth, Storage if needed, development dashboard.
Food data	USDA FoodData Central + suitable Indian food composition data + Open Food Facts where appropriate	Authoritative or practical nutrition datasets; normalize into the application database.
Deployment	Vercel for frontend; Railway/Render or equivalent for FastAPI	Simple deployment path for an MVP.
Mobile experience	Responsive PWA later	Installable app-like experience after core web flows are stable.

ARCHITECTURE PRINCIPLE

The user interface should never depend directly on third-party nutrition APIs for every interaction. Normalize/import trusted nutrition data into the application database, then let the backend serve the application. This improves performance, consistency, testing, and control over servings.

DATABASE MODEL TARGET

Table	Important fields	Purpose
users	id, email, created_at	Authentication identity; can be provided by Supabase Auth.
user_profiles	user_id, name, age, gender, height_cm, weight_kg, activity_level, goal	Profile and calculator inputs.
daily_targets	user_id, effective_date, calorie_target, protein_g, carbs_g, fat_g	Stores goal snapshots so historical days remain understandable even after a goal changes.
foods	id, name, category, source, external_id, brand, default_unit	Searchable normalized food catalog.
food_nutrition	food_id, basis_grams, calories, protein_g, carbs_g, fat_g, fiber_g, sugar_g, sodium_mg, micronutrients...	Nutrition values normalized to a defined basis, typically per 100 g.
servings	id, food_id, label, grams, unit_type, quantity	Human-friendly quantities such as 1 egg, 1 roti, 1 cup, 50 g.
meals	id, user_id, meal_type, consumed_at, date	Breakfast/lunch/dinner/snack container for a date.
meal_items	id, meal_id, food_id, serving_id, quantity, grams, nutrition_snapshot...	Actual food logged. Snapshot calculated nutrition so later catalog edits do not rewrite history unexpectedly.
favorites	user_id, food_id	Fast access to common foods.
custom_foods	user_id, name, serving data, nutrition data	User-created foods/recipes in a later phase.
weight_logs	user_id, date, weight_kg	Optional progress tracking in a later phase.

NUTRITION AND CALCULATOR LOGIC

BMR / TDEE

Use the Mifflin-St Jeor equation for the initial calculator. Treat the result as an estimate. Keep the calculation logic in a tested, isolated backend/service module so it is easy to verify and change later.

```
Male BMR    = 10 × weight_kg + 6.25 × height_cm - 5 × age_years + 5
Female BMR  = 10 × weight_kg + 6.25 × height_cm - 5 × age_years - 161
TDEE       = BMR × selected activity factor
```

The UI should present maintenance calories first, then optional goal-adjusted targets. Do not imply that the result is a medical prescription.

Nutrition scaling

```
For a food stored per 100 g:
scaled_nutrient = nutrient_per_100g × selected_grams / 100
```

For serving-based foods, map the selected serving to grams, then use the same normalization calculation. If a food uses a non-weight unit, keep an explicit gram-equivalent in the serving record.

UI / UX DESIGN SYSTEM

- Mobile-first layout with a bottom navigation bar on phone-sized screens.
- Use a calm health/productivity aesthetic: light background, white cards, green primary actions, subtle blue/teal supporting accents.
- Use rounded cards and controls with consistent corner radius and spacing.
- The calorie progress visualization is the visual anchor of the dashboard: circular or ring-style progress with a clear numeric center, plus a horizontal progress bar for compact contexts.
- Use clear macro progress bars for protein, carbs, and fat.
- Use large, touch-friendly controls; avoid tiny text and tiny hit areas.
- The Add Food flow should be no more than a few taps for common foods: Search -> Food -> Quantity -> Meal -> Save.
- Use familiar food icons/photography only if legally and technically appropriate; do not depend on decorative images to communicate data.
- Keep information dense enough for a tracker but not visually crowded.

Reference screens

Screen	Required UI elements
Landing	Value proposition, Get Started, Log In, simple feature summary.
Onboarding	Progress indicator, age, gender, height, weight, activity, goal, validation.
Goal result	Maintenance calories, goal target(s), daily macros, Continue button.
Dashboard	Date, calorie ring, consumed/target/remaining, macro bars, meal list, Add Food.
Add Food	Search field, recent/favorites filters, search results with basic nutrition preview.

Quantity	Predefined serving chips/buttons, unit-aware labels, custom quantity input.
Nutrition details	Calories, macros, fiber, key micros, selected quantity, Add to Meal.
Meal selector	Breakfast, Lunch, Snacks, Dinner; current-day context.
Calendar	Month grid, month navigation, data indicators, selected-date daily totals.
Past day	Calories, progress, macros, meals, edit controls subject to phase.
Profile	Personal data, goals, favorites, custom foods, settings, logout.

PHASE-BY-PHASE IMPLEMENTATION PLAN

PHASE 0 - Project foundation and repository rules

Goal: Establish the application skeleton, development conventions, documentation files, and a working local development environment without implementing the full product yet.

Build tasks

- Inspect the existing repository. If no project exists, create the agreed Next.js/TypeScript foundation and a sensible monorepo or app structure for Next.js + FastAPI.
- Create README.md and EXPLANATION.md immediately.
- Create .env.example with placeholders only.
- Set up formatting/linting/type-checking scripts and a basic test setup appropriate to the chosen stack.
- Create an initial design token layer for spacing, radius, typography, surface colors, primary green accent, muted text, borders, success/warning/error states.
- Create reusable layout primitives and an AppShell pattern, but keep business logic minimal.
- Create a basic landing page matching the visual direction from the reference flowchart.
- Add a phase status table to README.md.

Definition of done

- The app starts locally without errors.
- Landing page is responsive on a phone viewport.
- README.md and EXPLANATION.md exist and are understandable.
- Lint/typecheck/build/test commands are documented and passing where applicable.
- No secrets are committed.

Required documentation update

- README.md: add the completed phase to the progress tracker, document new setup/commands/env vars, and note how to run and verify the current version.
- EXPLANATION.md: add a plain-English explanation of what was added, how the data/UI flow works, key implementation decisions, and any limitations or assumptions.

PHASE 1 - UI shell and navigation

Goal: Build the reusable mobile-first application shell and the main navigation structure without connecting real data.

Build tasks

- Implement bottom navigation for Home, Log/Add Food, Calendar, Profile.
- Create page shells for Dashboard, Add Food, Calendar, Profile and placeholder states for unavailable functionality.
- Implement shared header/date patterns and responsive desktop layout.
- Implement reusable Card, Button, Input, ProgressBar, CalorieRing, EmptyState, LoadingState, ErrorState, MealCard and MacroBar components.
- Match the reference visual hierarchy: calorie visualization as the dashboard anchor, macro bars below it, meal cards, strong Add Food action.
- Keep mock data clearly separated from production data and label it as mock/demo data in code.

Definition of done

- All primary routes/pages are reachable.
- Navigation works on mobile and desktop.
- Components are accessible and reusable.
- No fake data is presented as real user data outside explicit development fixtures.

Required documentation update

- README.md: add the completed phase to the progress tracker, document new setup/commands/env vars, and note how to run and verify the current version.
- EXPLANATION.md: add a plain-English explanation of what was added, how the data/UI flow works, key implementation decisions, and any limitations or assumptions.

PHASE 2 - Supabase authentication and user profile

Goal: Add real authentication and user profile storage so users can have private, persistent accounts.

Build tasks

- Set up Supabase Auth integration using the supported authentication approach for the project.
- Create user profile storage linked to the authenticated user.
- Implement sign up, log in, log out, session persistence, and protected routes.
- Create onboarding/profile form for age, gender, height, weight, activity level, and goal.
- Validate inputs on both client and server.
- Handle unauthenticated, session-loading, and auth-error states.
- Document Supabase setup and local environment variables.

Definition of done

- A new user can register and log in.
- Authenticated user can save and reload profile data.
- A different user cannot read another user's profile data.
- Protected routes behave correctly after refresh and logout.

External dependency / user action

- The user must create or open a Supabase project and provide only the public project URL/anon key through the local environment mechanism recommended by Supabase/Next.js. Do not request or store a service-role secret in the frontend.

- If email confirmation or OAuth is enabled, explain the additional dashboard settings required and stop for the user if manual configuration is necessary.

Required documentation update

- README.md: add the completed phase to the progress tracker, document new setup/commands/env vars, and note how to run and verify the current version.
- EXPLANATION.md: add a plain-English explanation of what was added, how the data/UI flow works, key implementation decisions, and any limitations or assumptions.

PHASE 3 - Calorie calculator and daily targets

Goal: Implement BMR/TDEE calculation, goal handling, and persistent daily calorie/macro targets.

Build tasks

- Implement a tested BMR calculator service using Mifflin-St Jeor.
- Implement activity multipliers as explicit configuration/data, not magic numbers scattered across the codebase.
- Implement maintenance calorie calculation and goal-adjusted targets.
- Implement initial macro target logic and clearly document its assumptions.
- Persist daily target snapshots so that historical days can retain an understandable target even when a user changes goals later.
- Build the onboarding result screen with maintenance calories, goal target, and macro targets.
- Add unit tests for edge cases and normal cases.

Definition of done

- Calculator produces deterministic results for known test inputs.
- Inputs have sensible validation limits.
- Dashboard can display the saved target.
- Changing a future/current goal does not rewrite old historical target snapshots unintentionally.

Required documentation update

- README.md: add the completed phase to the progress tracker, document new setup/commands/env vars, and note how to run and verify the current version.
- EXPLANATION.md: add a plain-English explanation of what was added, how the data/UI flow works, key implementation decisions, and any limitations or assumptions.

PHASE 4 - Food database and nutrition import pipeline

Goal: Create the application food catalog, nutrition schema, serving model, and a repeatable import/normalization pipeline.

Build tasks

- Create foods, food_nutrition, and servings tables/schema with source and external ID metadata.
- Define nutrition basis clearly, preferably per 100 g for weight-normalized data.
- Create a seed/import mechanism that can ingest a controlled dataset or API export and normalize field names and units.
- Handle duplicates and source attribution.
- Create a small verified seed dataset for development using a mix of common foods and Indian foods.
- Do not insert guessed or fabricated nutrition data.

- Add database indexes suitable for food name search and user-specific queries.
- Expose backend endpoints for food search and food detail.

Definition of done

- A known set of foods can be searched locally through the backend.
- Serving definitions map correctly to grams where applicable.
- Nutrition units are consistent and documented.
- Import can be repeated without uncontrolled duplication.

External dependency / user action

- If a USDA API key or data download account is required for the chosen import route, stop and tell the user exactly how to obtain it and where it belongs.
- For Indian food composition data, use a legally/technically appropriate source and document source/attribution requirements before importing it.

Required documentation update

- README.md: add the completed phase to the progress tracker, document new setup/commands/env vars, and note how to run and verify the current version.
- EXPLANATION.md: add a plain-English explanation of what was added, how the data/UI flow works, key implementation decisions, and any limitations or assumptions.

PHASE 5 - Food search, quantity selection, and nutrition details

Goal: Build the complete user-facing food logging selection flow using real database data.

Build tasks

- Implement fast food search with debounce/loading/error/empty states.
- Show recent and favorite foods where data exists, while keeping fallback behavior for new users.
- Build quantity selection with practical predefined options and a custom quantity control.
- Support unit-aware serving labels: grams, ml, pieces, eggs, rotis, cups, etc. Store a gram equivalent where needed.
- Calculate scaled nutrition based on selected quantity using backend-calculated or validated formulas.
- Show calories, protein, carbs, fat, fiber and available micronutrients.
- Let the user review the exact selected food + quantity before adding it to a meal.

Definition of done

- The user can search a real food, choose a quantity, and see correctly scaled nutrition.
- Custom quantities validate correctly.
- The same UI pattern works for different serving units.
- No nutrition values are produced from hard-coded guesses in the frontend.

Required documentation update

- README.md: add the completed phase to the progress tracker, document new setup/commands/env vars, and note how to run and verify the current version.
- EXPLANATION.md: add a plain-English explanation of what was added, how the data/UI flow works, key implementation decisions, and any limitations or assumptions.

PHASE 6 - Meal logging and daily calorie progress

Goal: Persist meals and items, calculate daily totals, and make the dashboard update in real time after logging food.

Build tasks

- Create meals and meal_items persistence with date/time and meal type.
- Build meal selection: Breakfast, Lunch, Snacks, Dinner.
- Save a nutrition snapshot on each meal item so historical entries remain auditable if catalog data changes.
- Implement daily aggregation for calories, protein, carbs, fat, fiber and available micros.
- Implement the calorie ring/progress bar showing consumed, target, and remaining/over target states.
- Implement protein/carbs/fat progress bars.
- Implement meal cards showing calories and allow opening the meal for details.
- Add food deletion/edit behavior appropriate to the phase.

Definition of done

- Adding a food changes the dashboard totals for the selected date.
- Removing an item decreases totals correctly.
- Over-target calories are handled gracefully.
- Daily totals match the sum of meal-item nutrition snapshots.
- User A cannot read or modify user B's meals.

Required documentation update

- README.md: add the completed phase to the progress tracker, document new setup/commands/env vars, and note how to run and verify the current version.
- EXPLANATION.md: add a plain-English explanation of what was added, how the data/UI flow works, key implementation decisions, and any limitations or assumptions.

PHASE 7 - Calendar and historical daily views

Goal: Give users a reliable way to go back to previous days and inspect or edit logged nutrition.

Build tasks

- Implement monthly calendar with previous/next month and Today controls.
- Mark dates with logged nutrition data.
- Optionally show target-adherence status using subtle indicators, not overwhelming colors.
- Implement selected-date detail view with calories, progress, macros and meals.
- Implement date-specific meal editing/deletion while preserving historical nutrition snapshots.
- Add monthly summary: average calories, logged days, target adherence, and optional average macros.
- Optimize date-range queries and avoid loading unnecessary historical records.

Definition of done

- User can open any date with data and see accurate totals.
- Past day edits update that day without affecting other dates.
- Calendar navigation remains responsive on mobile.
- No timezone-related date shifts occur for meal dates in the supported user timezone.

Required documentation update

- README.md: add the completed phase to the progress tracker, document new setup/commands/env vars, and note how to run and verify the current version.
- EXPLANATION.md: add a plain-English explanation of what was added, how the data/UI flow works, key implementation decisions, and any limitations or assumptions.

PHASE 8 - Favorites, recently eaten, and custom foods/recipes

Goal: Reduce the effort needed for repeated meals and support homemade foods where standardized data is insufficient.

Build tasks

- Implement favorite food toggling and favorite list.
- Implement recently eaten food list ordered by recent use.
- Implement custom food creation with serving and nutrition data.
- Implement a recipe/custom meal model if the data model supports it cleanly; otherwise keep it as a custom food first and document the tradeoff.
- Allow editing/deleting user-created foods without affecting historical meal nutrition snapshots.
- Add clear labels distinguishing system foods from user-created foods.

Definition of done

- Common foods can be added in very few taps.
- Favorites persist per user.
- Custom food values are validated and stored safely.
- Historical logs do not unexpectedly change when a custom food is edited.

Required documentation update

- README.md: add the completed phase to the progress tracker, document new setup/commands/env vars, and note how to run and verify the current version.
- EXPLANATION.md: add a plain-English explanation of what was added, how the data/UI flow works, key implementation decisions, and any limitations or assumptions.

PHASE 9 - Profile, goals, weight tracking, and insights

Goal: Complete the personal tracking area and add simple historical insights without overcomplicating the app.

Build tasks

- Build profile and goals editing screens.
- Allow weight logging by date.
- Add a simple weight trend visualization.
- Add a monthly nutrition summary using historical calendar data.
- Ensure changing profile inputs triggers a documented target recalculation workflow without rewriting historical facts.
- Add settings for sensible preferences such as units where supported.

Definition of done

- Profile changes persist safely.

- Weight history is date-based and graphable.
- Historical nutrition remains stable and interpretable.
- Insights use real stored data and clearly indicate when there is insufficient data.

Required documentation update

- README.md: add the completed phase to the progress tracker, document new setup/commands/env vars, and note how to run and verify the current version.
- EXPLANATION.md: add a plain-English explanation of what was added, how the data/UI flow works, key implementation decisions, and any limitations or assumptions.

PHASE 10 - Mobile/PWA polish, accessibility, testing, and production readiness

Goal: Harden the application for real phone use and prepare a deployable MVP.

Build tasks

- Audit all primary screens at common mobile widths and at least one desktop width.
- Add PWA support if the core web application is stable and the chosen approach is appropriate.
- Improve loading performance, caching and database query efficiency where justified.
- Run full lint, typecheck, test and production build checks.
- Add integration/end-to-end tests for the critical journey: sign in -> set goal -> search food -> choose quantity -> add meal -> verify dashboard -> open calendar -> open prior day.
- Perform accessibility checks for keyboard/focus/labels/contrast and touch target sizing.
- Add clear error handling and safe retry behavior for network/database failures.
- Document deployment steps and production environment variables without exposing secrets.

Definition of done

- Critical user journey passes from a clean account.
- Production build succeeds.
- Application is usable on phone-sized screens.
- No known critical security or data-isolation issue remains for the MVP.

External dependency / user action

- If deployment requires a Vercel/Railway/Render account, stop before creating or linking paid resources and ask the user to create/connect the required project.
- If a custom domain is desired, treat domain/DNS setup as a separate explicit user action unless the user has already provided a connected deployment environment.

Required documentation update

- README.md: add the completed phase to the progress tracker, document new setup/commands/env vars, and note how to run and verify the current version.
- EXPLANATION.md: add a plain-English explanation of what was added, how the data/UI flow works, key implementation decisions, and any limitations or assumptions.

OPTIONAL FUTURE PHASES - DO NOT BUILD DURING MVP

- Barcode scanning for packaged foods using Open Food Facts or another suitable provider.

- Photo-based food recognition using a vision model, with user confirmation and manual quantity correction. The model should identify candidate foods/servings; it should not invent nutrition values.
- Food suggestions and meal planning based on user history, implemented only after privacy and consent requirements are clearly defined.
- Offline-first logging and sync.
- Export to CSV/PDF.
- Advanced analytics and trends.
- Native mobile apps if the PWA no longer meets product needs.

EXTERNAL ACCOUNT / DEPENDENCY CHECKLIST

Dependency	When needed	What Antigravity should ask the user for	What it must NOT do
Supabase	Phase 2	Ask user to create/select project and configure supported public project variables in .env.local or equivalent.	Never fabricate credentials or expose service-role secrets in frontend code.
USDA FoodData Central	Phase 4 if API/import requires it	Ask user to obtain the required API key or choose a data-download route; explain exact environment variable name and purpose.	Never paste a fake key or silently create an external account.
Indian food data source	Phase 4	Ask which legally usable source is available if more than one is viable; document attribution/licensing.	Do not scrape or import restricted data without checking allowed usage.
Open Food Facts	Phase 4 or later	No account may be needed for basic public data access; verify current provider requirements before relying on it.	Do not assume current API limits or terms if they have changed; document the exact integration used.
Deployment provider	Phase 10	Ask user to create/connect a project and configure production environment variables.	Do not create paid resources without explicit approval.
Custom domain	Optional	Ask user to provide domain or connect their domain provider.	Do not modify DNS without explicit user action/authorization.

README.md AND EXPLANATION.md CONTRACT

README.md must always contain

- Project purpose and one-paragraph product description.
- Current phase and completed phases.
- Feature list with completed/in-progress/planned status.
- Tech stack.
- Folder/project structure overview.
- Local setup instructions.
- Environment variables with safe placeholders.
- Commands for dev, lint, typecheck, tests, and build.

- Database setup/migrations and seed/import commands.
- External accounts/services required and setup links when appropriate.
- Known limitations and next phase.

EXPLANATION.md must always contain

- Plain-English product overview.
- End-to-end user flow.
- Frontend architecture and why the main components exist.
- Backend architecture and API responsibilities.
- Database model with an understandable explanation of each major table.
- How calorie/BMR/TDEE calculations work.
- How food quantities are converted into nutrition values.
- How meal totals and dashboard progress are computed.
- How the calendar retrieves historical days.
- Data privacy and user isolation explanation.
- What Antigravity changed in the current phase and why it chose that approach.
- Known tradeoffs, assumptions, and future improvements.

DOCUMENTATION QUALITY BAR

Assume the reader is a student/developer who did not see the implementation session. Explain what the code does, why it exists, and how to verify it. Prefer diagrams, tables, short examples, and plain language over unexplained jargon.

HOW TO REQUEST EACH PHASE FROM ANTIGRAVITY

Use a separate instruction at the beginning of each work session. Examples:

Execute Phase 0 only. Follow the Master Prompt rules. Inspect the repo first, implement the phase, verify it, update README.md and EXPLANATION.md, summarize what changed, identify any external action I need to take, and STOP.

Execute Phase 1 only. Review the current repository and documentation before coding. Reuse existing work. Implement only Phase 1 from the master specification, then verify, update README.md and EXPLANATION.md, report completion and STOP.

Execute Phase 4 only. Before coding, verify whether an external nutrition data account/API key is required. If user action is required, stop and explain the exact setup I need to do. Otherwise implement Phase 4, test it, document it, and STOP.

FINAL PRODUCT ACCEPTANCE CHECKLIST

- User can create an account and securely access only their own data.
- User can enter profile/goal information and receive an understandable calorie target.
- User can search a food by name.
- User can select a practical quantity or custom quantity.
- Nutrition values scale correctly to the chosen quantity.
- User can add food to Breakfast/Lunch/Snacks/Dinner.
- Dashboard shows daily calories consumed, target, remaining/overage, and progress bar/ring.
- Dashboard shows protein, carbs and fat progress.
- User can open the calendar and navigate to previous days.

- Previous-day details show accurate meals and nutrition.
- Historical data remains stable when the food catalog or user goals change.
- UI is polished and usable on a phone.
- README.md and EXPLANATION.md are current after every phase.
- Production build passes and critical flows are tested.
- No secret credentials are committed to the repository.

END OF MASTER PROMPT